

antoineview

A technical reference for the HTML, CSS and modern web-platform features powering the portfolio.

Covers all tags and CSS properties used in `index.html`, the eight project pages, `css/style.css` and `css/article.css`. Special focus on scroll-driven animations, SVG path animation, and sticky positioning.

Contents

- | | |
|-------------------------------------|---|
| 1. HTML tags | ★ 5. SVG path animation |
| 2. CSS properties by category | ★ 6. Sticky & fixed positioning |
| ★ 3. Section-by-section walkthrough | 7. At-rules (@font-face, @media, @keyframes, @supports) |
| ★ 4. Scroll-driven animations | 8. Recurring techniques |

1. HTML tags

Every tag that appears in `index.html` or a project page. Beyond the standard semantics, each entry notes how this project actually uses the tag.

Document scaffolding

<code><!doctype html></code>	HTML5 document declaration. Must be the very first line — without it browsers fall into quirks mode and many layout rules behave differently.
<code><html lang></code>	Root element. <code>lang="en"</code> tells screen readers and search engines the document language.
<code><head></code>	Container for metadata that isn't rendered: title, meta tags, fonts, stylesheets.
<code><meta></code>	Single-attribute metadata. Used here for character encoding (<code>charset</code>), responsive viewport (<code>name="viewport"</code>), and SEO description.
<code><title></code>	Browser tab label and default bookmark name.
<code><link></code>	External resource. Two uses: <code>rel="stylesheet"</code> for CSS, and <code>rel="preload" as="font"</code> to prefetch web fonts before the first paint so no FOUT (Flash Of Unstyled Text).
<code><body></code>	Holds everything that renders.

Semantic structure

<code><header></code>	Top region. Holds the fixed nav and its progressive-blur backdrop.
<code><nav></code>	Primary navigation. Contains the site links (About / Projects / Music).
<code><main></code>	The single main landmark for the page. There must be exactly one. Screen readers can jump straight to it.
<code><section></code>	A thematic grouping with its own heading. Hero, About, Projects, Music, etc.
<code><article></code>	Self-contained content; used here for each <code>.paper</code> sheet on a project page.
<code><footer></code>	Closing region. (Site footer.)

`<figure> /
<figcaption>`

An image with its caption, kept together semantically. Used for every polaroid on project pages.

Content & text

`<h1>`

Top-level heading. One per page (visually hidden on the homepage because the SVG name takes its place).

`<h2>`

Section heading: About, Projects, Music, Currently working on...

`<p>`

Paragraph. The atomic block of body copy.

`<ul> / <ol>`

Unordered / ordered list. The nav links are an `<ul>` ; the projects timeline is an `<ol>` (the order matters, it's chronological).

`<li>`

One list item. Each project is one `<li>` .

`<a href>`

Hyperlink. The internal jumps (`#about`) plus the cassettes that link to project pages.

`<br>`

Explicit line break inside a paragraph (used sparingly in the About text).

`<b>`

Stylistically bold inline text without semantic emphasis (used inside `.meta` on music sleeves).

Media & presentation

`<img src alt>`

Raster/vector image. `alt` is mandatory — empty `alt=""` for purely decorative images (camera, portrait), descriptive `alt` for content images (project previews).

`<div>`

Generic block grouping with no semantics. Used for layout shells (`.about-stage` , `.nav-blur` , etc.).

`<span>`

Generic inline grouping. Used for the progressive blur layers — each `<span>` carries inline CSS custom properties (`--b` , `--t` , `--e` , `--m`) that the stylesheet reads.

SVG (used in the animated name)

`<svg viewBox>`

Inline vector graphic. `viewBox` defines the internal coordinate system; the SVG then scales fluidly via the `.draw-name { width: min(60vw, 32.5rem) }` rule.

`<g>`

Group element. Used to apply a single SVG `filter` (the glow halo) to all strokes at once.

`<path d>`

The geometry. Each letter stroke is one `<path>` with a vector `d` attribute and a `pathLength="1"` normalisation (see §4).

`<defs>`

Reusable definitions. The gradients, filter and clip paths sit inside `<defs>` ; nothing inside is rendered directly, only referenced by id.

`<filter>`

SVG filter primitive container. Used for the soft glow behind each stroke.

`<feGaussianBlur>`,
`<feMerge>`,
etc.

Filter primitives composing the glow.

`<linearGradient>
/ <stop>`

Defines the colour gradients on individual strokes.

2. CSS properties by category

Properties grouped by what they actually do, with the concrete role each plays in the site.

Layout: block model

Property	What it does
<code>box-sizing</code>	Switches width/height calculation to include padding+border. Set globally to <code>border-box</code> so all our widths "mean what they say."
<code>display</code>	Sets the formatting context. We use <code>grid</code> , <code>flex</code> , <code>block</code> , <code>inline</code> and <code>none</code> depending on the element.
<code>width</code> / <code>height</code>	Explicit box dimensions. Often paired with <code>min()</code> , <code>max()</code> or <code>clamp()</code> to stay responsive.
<code>max-width</code> / <code>min-height</code>	Soft caps. <code>main { max-width: 45rem }</code> is the central content column; <code>.hero-pin { min-height: 100vh }</code> reserves a viewport-height pinning area.
<code>aspect-ratio</code>	Locks width-to-height ratio. The vinyl <code>.disc</code> and the polaroid square images use it.
<code>margin</code> / <code>margin-inline</code> / <code>margin-block</code>	Outer space. Logical-axis variants (<code>-inline</code> , <code>-block</code>) flow with writing direction.
<code>padding</code> / <code>padding-inline</code> / <code>padding-block</code>	Inner space, same logical-axis pattern.
<code>border</code> / <code>border-radius</code> / <code>border-top</code>	Edge decoration; <code>border-radius</code> rounds the portrait corners.

Layout: positioning

Property	What it does
<code>position</code>	<code>static</code> (default), <code>relative</code> , <code>absolute</code> , <code>sticky</code> , <code>fixed</code> . See §5.
<code>top</code> / <code>right</code> / <code>bottom</code> / <code>left</code>	Offsets for non-static positioning.
<code>inset</code> / <code>inset-inline</code>	Shorthand for all four (or two) offsets at once. <code>inset: 0</code> = stretch to all edges.
<code>z-index</code>	Stacking order in a positioned context. The about layers (portrait 1 → veil 2 → camera 3 → text 4) live in the same grid cell, <code>z-index</code> controls who paints on top.
<code>isolation</code>	Forces a new stacking context. Used on <code>.cover</code> to keep its shadows from bleeding into the sleeves grid.
<code>overflow</code> / <code>overflow-x</code>	<code>visible</code> (default), <code>hidden</code> , <code>clip</code> , <code>scroll</code> , <code>auto</code> . <code>clip</code> blocks scrolling without making the element a scroll container — important so descendant <code>position: sticky</code> still works.

Layout: flexbox & grid

Property	What it does
<code>flex-direction</code>	<code>row</code> or <code>column</code> . The article pages use <code>flex-direction: column</code> with <code>align-items: center</code> to stack papers and polaroids.

<code>justify-content</code>	Main-axis distribution.
<code>align-items</code> / <code>align-self</code>	Cross-axis alignment per container / per child.
<code>justify-self</code>	Per-item main-axis alignment in a grid cell.
<code>place-items</code>	Shorthand for <code>align-items</code> + <code>justify-items</code> .
<code>gap</code>	Spacing between grid/flex items (replaces ad-hoc margins between children).
<code>grid-template-columns</code>	Defines column tracks. We use <code>1fr 1fr</code> (projects), <code>repeat(auto-fit, minmax(14rem, 1fr))</code> (hero), and a single <code>var(--about-w)</code> track (about).
<code>grid-template-areas</code>	Lets multiple grid items share one named cell. The about portrait/veil/camera/text all declare <code>grid-area: stage</code> so they overlap in the same cell, ordered by z-index.
<code>grid-area</code>	Places an item in a named area.
<code>order</code>	Reorders flex/grid items independently of DOM order. Mobile project layout uses <code>.frame { order: -1 }</code> to pull the TV above the timeline.

Typography

Property	What it does
<code>font-family</code>	Webfont stack. Custom BBB Karrik + system fallback.
<code>font-size</code>	Often a <code>clamp(MIN, FLEX, MAX)</code> for fluid type that respects a floor and ceiling.
<code>font-weight</code> / <code>font-style</code>	Standard weight/italic switches.
<code>line-height</code>	Leading (vertical rhythm).
<code>letter-spacing</code>	Tracking. The article title uses <code>0.08em</code> for a touch of poster look.
<code>text-align</code> / <code>text-transform</code>	Alignment and uppercase transforms.
<code>text-decoration</code>	Set to none on links globally; we let other styles signal interactivity.
<code>color</code>	Text colour. Driven from <code>--color-fg</code> tokens.
<code>white-space</code>	nowrap on the cassette label keeps it on one line.
<code>-webkit-font-smoothing</code>	macOS-only switch for antialiased font rendering.
<code>-webkit-text-size-adjust</code>	Stops mobile Safari auto-scaling text when the user rotates the device.

Colour, fills & backgrounds

Property	What it does
<code>background</code>	Shorthand for all background sub-properties. Most uses here are <code>linear-gradient(...)</code> for the veils and nav blur.
<code>background-color</code>	Solid fill. Used for the polaroid paper colour <code>#faf7ed</code> .

background-image / -size / -position / -repeat	Used by the vinyl PNGs on music sleeves and the parchment letter_paper.png on article pages.
opacity	Whole-element transparency. The .tv-pic images live at opacity 0 until a hover swap.

Filters & effects

Property	What it does
filter	SVG-style filter chain. The polaroid images get <code>sepia(0.22) saturate(0.85) contrast(0.94) brightness(1.02)</code> for the faded-print look.
backdrop-filter / -webkit-backdrop-filter	Filters what's behind the element rather than the element itself. The progressive blur stacks on the nav, edge-blur and about veil are all <code>backdrop-filter: blur(var(--b))</code> .
mask-image / -webkit-mask-image	Uses a gradient as a luminance/alpha mask so the blur fades in and out smoothly. Each layer reads its own <code>--t/--e</code> or <code>--m</code> via CSS variables.
clip / clip-path	Geometric clipping. Used on the .visually-hidden screen-reader utility.
box-shadow	Layered shadows on the polaroids and music sleeves give them depth.

Transforms & animation

Property	What it does
transform	2D/3D transforms. We use <code>translateX(-50%)</code> for centering tricks, <code>rotate()</code> for the polaroid tilt and the article pen flourish.
translate / rotate	Modern individual transform properties. Used in the hero <code>@keyframes drift</code> .
transform-origin	Sets the pivot point for rotation/scale. The pen rotates around its right center.
transition	Tweens declarative property changes (hover states on sleeves, TV picture cross-fades).
animation	Triggers a <code>@keyframes</code> sequence. See §3 for scroll-driven, plus the hero drift, vinyl spin, etc.
animation-name / -duration / -delay / -timing-function / -fill-mode	The longhand controls.
animation-play-state	paused/running. The vinyl .disc spin is paused by default and switched to running on hover via <code>:has()</code> .
animation-timeline	Bind an animation to a scroll/view timeline instead of wall-clock time. See §3.
animation-range	Slice of the timeline an animation occupies. See §3.
view-timeline	Names a timeline that progresses as the element enters and leaves the viewport. See §3.

`will-change`

Hint to the browser ("I'm about to animate this"). Set on `.hero-pic { will-change: translate }` so it gets a GPU layer up front.

SVG-specific properties (used in the name animation)

Property	What it does
<code>stroke-dasharray</code>	Defines the dash pattern along a stroke. Set to 1 with <code>pathLength="1"</code> = "one dash equal to the whole path length." See §4.
<code>stroke-dashoffset</code>	Pushes the dash pattern along the stroke. Going from 1 to 0 reveals the stroke. See §4.
<code>visibility</code>	hidden until the animation fires so the path doesn't flash at <code>dashoffset: 1</code> .

Pointer / interaction

Property	What it does
<code>cursor</code>	Set globally to <code>pointer</code> for <code>a[href]</code> , buttons and <code>[role="button"]</code> so every clickable thing gets the pointing hand.
<code>pointer-events</code>	<code>none</code> lets clicks pass through purely-visual layers (the nav blur, the about camera, the edge-blur).
<code>user-select</code> / <code>-webkit-user-drag</code>	Disable selection / drag on decorative images (portrait, camera, TV frame).
<code>scroll-behavior</code>	<code>smooth</code> on <code>html</code> for in-page jumps (clicking About/Projects/Music in the nav).
<code>scroll-margin-top</code>	Offsets scroll-snap landing points. Every section reserves <code>2 × --space</code> so the fixed nav doesn't cover the heading on jump.

3. ★ Section-by-section walkthrough

The site is one long `<main>` made of five sections plus a fixed header and a footer. Each section uses a different mix of layout primitives. This section explains, in order, what the HTML looks like and what each piece does.

3.1 `<header class="site-header">` — the fixed nav

```
<header class="site-header">
  <div class="nav-blur" aria-hidden="true">
    <span style="--b: 2px; --t: 45%; --e: 100%"></span>
    <span style="--b: 4px; --t: 38%; --e: 88%"></span>
    <!-- ... 4 more progressive layers -->
  </div>
  <nav class="nav">
    <a class="brand" href="#top">antoineview</a>
    <ul class="nav-links">
      <li><a href="#about">About</a></li>
      <li><a href="#projects">Projects</a></li>
      <li><a href="#music">Music</a></li>
    </ul>
  </nav>
</header>
```

Two children: a decorative `.nav-blur` backdrop and the real `<nav>`. The `aria-hidden="true"` tells screen readers to skip the blur (it's purely visual). The header itself is `position: fixed; top: 0; left: 0; right: 0` so it stays anchored to the viewport on every scroll. Each blur `<span>` carries its parameters as inline CSS variables (`--b` =radius, `--t` =mask solid-end, `--e` =mask transparent-end), so one CSS rule renders six different progressive blur layers.

3.2 `<section class="hero" id="top">` — the scroll-driven name

```
<section class="hero" id="top">
  <div class="hero-pin">
    <div class="hero-imgs" aria-hidden="true">
      
      <!-- 7 more drifting pictures -->
    </div>
    <div class="edge-blur" aria-hidden="true">...</div>
    <div class="edge-fade" aria-hidden="true"></div>
    <h1 class="visually-hidden">antoine</h1>
    <svg class="draw-name" viewBox="0 0 854 163">
      <path style="--i: 5" pathLength="1" d="..." />
      <!-- 23 more path strokes -->
    </svg>
    <p class="scroll-cue">scroll</p>
  </div>
</section>
```

Two-layer structure. The outer `.hero` is the *scroll runway* — it's 300vh tall when scroll-driven animations are supported, so you can scroll three viewports' worth while looking at it. The inner `.hero-pin` is `position: sticky; top: 0; height: 100vh` — it pins to the top of the viewport for the duration

of that runway. While the runway is in view, scrolling advances the SVG path animation (see §5) without moving the visible content.

What lives inside the pin.

- `.hero-imgs` : the drifting photographs. A CSS grid of images, each running a `@keyframes drift` animation on a 36-second loop with staggered `animation-delay`.
- `.edge-blur` & `.edge-fade` : progressive blur and a white gradient at the viewport edges, so the drifting photos dissolve into the page background rather than cutting off at a hard rectangle.
- `.visually-hidden <h1>` : a screen-reader-only "antoine" so the page has a real H1 even though the visible logotype is an SVG.
- `.draw-name` : the SVG itself — 24 paths drawing each letter stroke. See §5.

3.3 `<section id="about" class="about">` — sticky portrait + scrolling text

```
<section id="about" class="about">
  <h2 id="about-heading">About</h2>
  <div class="about-stage">
    
    <div class="about-veil">
      <span class="about-veil-white"></span>
      <span class="about-veil-blur" style="--b: 2px; --m: 100%"></span>
      <!-- 5 more progressive blur layers -->
    </div>
    
    <div class="about-text">
      <p>Hi! I'm Antoine</p>
      <!-- more paragraphs -->
    </div>
  </div>
</section>
```

The trick: four layers in the same grid cell.

```
.about-stage {
  display: grid;
  grid-template-columns: var(--about-w);
  grid-template-areas: "stage";
  justify-content: center;
  min-height: 220vh;
}
.about-portrait, .about-veil, .about-camera, .about-text {
  grid-area: stage; /* all four land in the same cell */
}
```

By declaring a single named area and putting all four elements into it, they overlap. `z-index` then controls who paints on top: portrait (1) → white veil (2) → camera (3) → text (4).

The scroll choreography. The `.about-stage` is 220vh tall (a long runway). Inside it:

- The **portrait** is `position: sticky; top: 25%`. It pins a quarter of the way down the viewport while the stage scrolls past.
- The **veil** and **camera** are in normal flow (no sticky). They scroll up off-screen as you advance.

- The **text** is in normal flow too, but with `padding-block-start: calc(var(--about-w) + 5vh)` so its first paragraph begins below the bottom of the camera. As the text scrolls up past the still-sticky portrait, it reads as a caption track over the trees.

The white veil hides the portrait *until* the camera (and the veil with it) has scrolled away. Then the portrait alone is left, with the text scrolling over it.

Full-bleed. The `.about` uses `width: 100vw; margin-inline: calc(50% - 50vw)` to break out of `main { max-width: 45rem }` and span the entire viewport — necessary because the camera and portrait want all the horizontal room they can get.

3.4 `<section id="projects" class="projects">` — sticky TV + scrolling timeline

```
<section id="projects" class="projects">
  <h2>Currently working on</h2>
  <a class="k7-link" href="projects/chrysalide.html">
    <div class="k7-full">
      
      
      
      <div class="player-front">...</div>
    </div>
  </a>
  <div class="projects-grid">
    <ol class="timeline">
      <li class="year"><span class="label">2026</span></li>
      <li class="proj" data-project="scc26">
        <a class="proj-link" href="projects/scc26.html">
          <span class="k7">
            
            <span class="k7-title">SCC 26</span>
          </span>
        </a>
      </li>
      <!-- more years and cassettes -->
    </ol>
    <div class="frame">
      <div class="frame-veil">...</div>
      <div class="projects-head"><h2>All Projects</h2></div>
      <div class="tv">
        
        
        <!-- one tv-pic per project, all at opacity 0 -->
        
      </div>
    </div>
  </div>
</section>
```

The "Currently working on" cassette. Wrapped in a single `<a>` so the whole composite is clickable. Hover effects (lift + drop shadow) are scoped to `.k7-full`; a `:has(.player-front:hover)` selector spins the axes only when the cover face is hovered, not the cassette body.

The timeline + TV. A two-column grid: `<ol class="timeline">` on the left (chronological — that's why it's an ordered list, not an unordered one), `.frame` on the right containing the TV. The `.frame` is

`position: sticky; top: var(--head-top)`, so the TV stays parked beneath the nav while you scroll down the cassette list. The `.tv-pic` images are stacked inside the TV at `opacity: 0`; on hover of any `.proj`, a `:has()` rule sets the matching `.tv-pic` to opacity 1, cross-fading the TV screen to that project's preview.

3.5 `<section id="music">` — vinyl sleeves

```
<section id="music">
  <h2>Music on repeat</h2>
  <ul class="sleeves">
    <li class="sleeve v-orange">
      <span class="disc-slot"><span class="disc"></span></span>
      <span class="cover">
        
      </span>
      <span class="meta"><b>Mydnight</b></span>
    </li>
    <!-- one li per album -->
  </ul>
</section>
```

Each `.sleeve` contains three stacked pieces: an absolutely-positioned `.disc-slot` (the vinyl record, initially hidden behind the cover), a `.cover` (the album art), and a `.meta` caption underneath. On hover the cover translates downward, the disc slides out and spins via `@keyframes spin`. Each `.sleeve` carries a colour modifier class (`v-orange`, `v-black`, etc.) that swaps in a different vinyl PNG. The `.sleeves` container uses `grid-template-columns: repeat(auto-fill, minmax(9.5rem, 1fr))` so the grid re-flows naturally with viewport width.

3.6 `<footer class="site-footer">`

Just a thin closing band with copyright text. `border-top: 1px solid #eee`, muted colour, centred.

3.7 Project article pages (e.g. `projects/scc26.html`)

```
<main class="article">
  <h1 class="article-title">SCC 26</h1>
  <article class="paper">
    <p>...</p>
    <p>...</p>
  </article>
  <figure class="polaroid polaroid--1">
    
    <figcaption>dashboard, the landing page</figcaption>
  </figure>
  <!-- more paper + polaroid pairs -->
  <article class="paper paper--last">
    <p>...</p>
    
  </article>
</main>
```

Article pages use a different stylesheet (`article.css`) layered on top of the shared `style.css`. The layout is a vertical stack of `<article class="paper">` sheets (textured paper background, two-column

rule decoration) interleaved with `<figure class="polaroid">` photographs that are rotated slightly (alternating `--l / --r` tilts) and hang over the previous paper via `margin-top: -11rem`. The final article carries a `.paper--last` modifier with a hand-drawn pen flourish (`.pen`) signing off the page.

Each `<figure> + <figcaption>` pair is a single semantic unit — the caption is bound to the image, which matters for screen readers.

4. ★ Scroll-driven animations

Used in: §3.2 (hero name draws as you scroll). Declares a timeline channel on §3.3 (about) for downstream effects.

The hero name draws itself as you scroll, and the about section advertises a timeline downstream effects can hook into. Both use the brand-new *Scroll-driven Animations* spec, which connects a regular `@keyframes` animation to scroll progress instead of wall-clock time.

The three pieces

- `view-timeline: --name block` — declares that *this element* exposes a timeline named `--name` that advances from `0` when the element first enters the viewport to `1` when it has just left, measured along the block axis (vertical for default writing modes).
- `animation-timeline: --name` — binds an animation to that timeline. Progress through the keyframes now follows scroll progress, not seconds.
- `animation-range: contain X% contain Y%` — restricts which slice of the timeline the animation occupies. `contain` means "the element is fully contained in the viewport," so `contain 0% contain 100%` runs the animation only while the element is fully on screen.

The hero name, in detail

```
/* Outer: 300vh-tall scroll runway exposes a timeline */
@supports (animation-timeline: view()) {
  .hero {
    height: 300vh;
    view-timeline: --hero block;
  }

  .hero-pin {
    position: sticky; top: 0; height: 100vh;
  }

  /* Each letter stroke binds to the same timeline */
  .draw-name path {
    animation-name: draw-stroke;
    animation-timeline: --hero;
    animation-duration: auto;           /* duration is the timeline */
    animation-delay: 0s;
    animation-timing-function: linear;
    animation-fill-mode: both;
    /* And each stroke gets its own 1/24 slice */
    animation-range:
      contain calc((var(--i) - 1) / 24 * 100%)
      contain calc((var(--i)) / 24 * 100%);
  }
}
```

The strokes are numbered 1–24 via an inline CSS variable `--i` on each `<path>`. Each stroke draws across its own equal 1/24 slice of the contain phase, so by the time the section has fully scrolled past, every letter has appeared in order.

Fallback for browsers without scroll-timeline

Outside the `@supports` block, the same keyframes run on a normal time-based animation with a per-stroke `animation-delay: calc((var(--i) - 1) * 170ms)`. Same visual result, just untethered from scroll. This is graceful degradation: the modern path is opt-in via feature query.

The about timeline

```
.about {  
  view-timeline: --about block;  
}
```

The about section declares a timeline named `--about` so future scroll-driven effects (e.g. crossfading the portrait, advancing parallax) can attach by name without re-measuring scroll position from JavaScript.

Mental model

Think of `view-timeline` as the element *publishing* its own progress as a 0→1 channel that anyone in the cascade can subscribe to. `animation-timeline` is the subscription. `animation-range` is a window filter on that subscription. The keyframes themselves don't change — they just play to a different clock.

This replaces a category of JavaScript that used to listen on `scroll`, compute positions, and write transforms every frame. It's declarative, off the main thread, and respects `prefers-reduced-motion`.

5. ★ SVG path animation

Each letter in the SVG name draws itself stroke-by-stroke. The technique combines three SVG/CSS features.

1. `pathLength`

Without `pathLength`, the dash pattern is in *user units* — so a long stroke and a short stroke would need different dash values to look the same. Setting `pathLength="1"` on the `<path>` tells the browser "pretend this path has length 1," normalising every stroke to the same length regardless of its actual geometry.

2. `stroke-dasharray` and `stroke-dashoffset`

A stroke can be drawn as a dashed line by setting `stroke-dasharray`. With `pathLength="1"` + `stroke-dasharray: 1`, you've drawn one dash exactly the length of the whole path: a single, solid line.

`stroke-dashoffset` shifts where that dash starts. At offset `1` (= one whole path length away), the dash is entirely off the end → invisible. At offset `0`, it's perfectly aligned → fully visible.

```
.draw-name path {
  stroke-dasharray: 1;
  stroke-dashoffset: 1; /* hidden */
  visibility: hidden;
  animation: draw-stroke 170ms linear forwards;
}

@keyframes draw-stroke {
  0% { stroke-dashoffset: 1; visibility: hidden; }
  100% { stroke-dashoffset: 0; visibility: visible; }
}
```

3. Ordering with a custom property

Each `<path>` is tagged inline with `style="--i: 8"` (1–24, the writing order). The CSS then derives both:

- **The fallback delay:** `animation-delay: calc((var(--i) - 1) * 170ms)` — letter `--i` waits its turn.
- **The scroll-timeline range:** `animation-range: contain calc((--i - 1)/24 * 100%) contain calc(--i/24 * 100%)` — letter `--i` occupies its slice of the scroll runway.

Why `visibility: hidden` at the start?

Subpixel anti-aliasing can leak a faint sliver of the stroke even at `dashoffset: 1`. Toggling `visibility` through the keyframes guarantees a clean start: nothing renders until the first frame where the dashoffset is meaningfully less than 1.

6. ★ Sticky & fixed positioning

"Sticky" is the most useful and most misunderstood positioning value in CSS. It's the engine behind every scroll choreography in this site, so it gets a deep dive.

The mental model

A sticky element behaves like `position: relative` *most* of the time. It flows with the document, takes up its natural space, and scrolls along with its parent. **But** when the scroll position would cause the element to leave the viewport on its *nearest scrolling ancestor*, the browser switches it to behave like `position: fixed` at the offset you specified (e.g. `top: 0`). The element appears to "stick" to that offset.

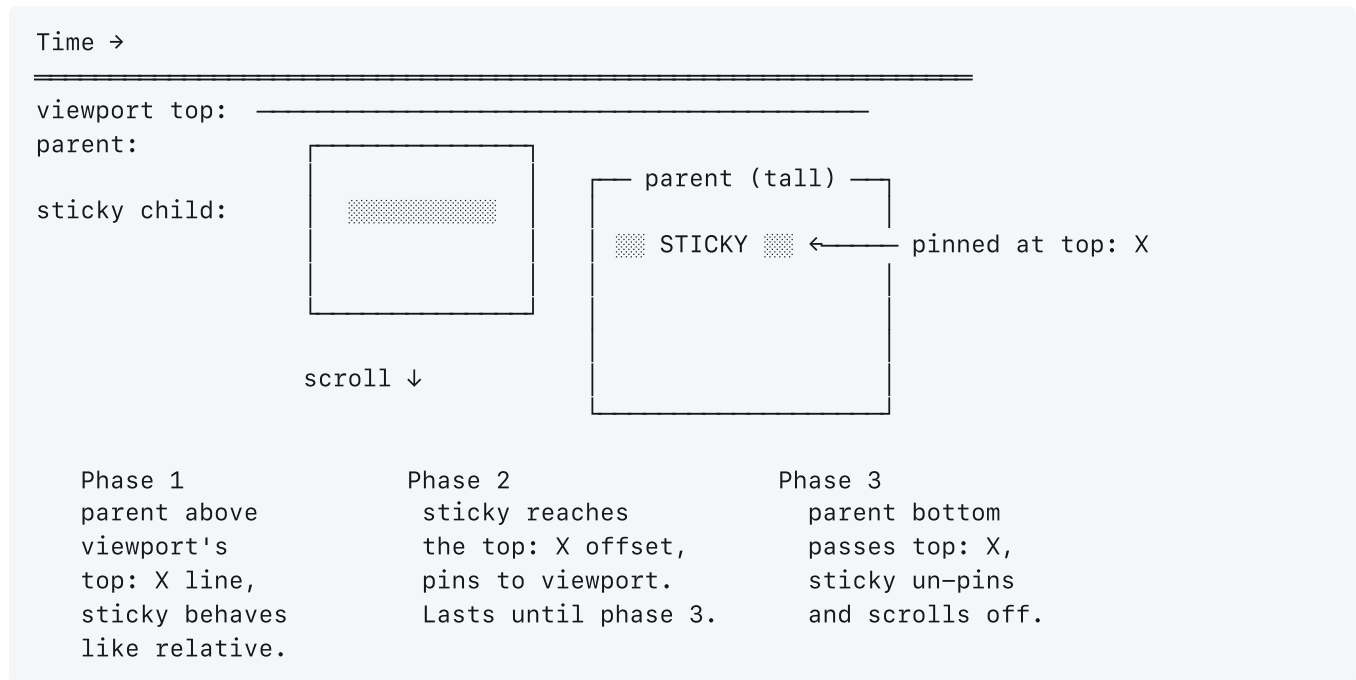
The pin lasts only as long as the element's *containing block* (its nearest ancestor with a layout role) is still visible. Once the bottom of the containing block scrolls past the sticky offset, the element un-pins and scrolls away with its parent.

The three required ingredients

1. **An offset.** You must declare at least one of `top`, `bottom`, `left`, `right`. Without one, sticky silently degrades to `position: relative` and nothing happens.

2. **A parent taller than the sticky element.** The parent needs scroll headroom for the element to stick within. If parent and child are the same height, they leave the viewport together — no sticking ever happens.
3. **No scroll-blocking ancestor.** Any ancestor with `overflow: hidden`, `overflow: auto`, or `overflow: scroll` becomes the new scrolling root for the sticky. If that ancestor doesn't itself scroll, the sticky inside it can't either.

A timeline of one sticky element



The three sticky uses in this site

1. The hero pin — pinning the whole hero for 300vh of scroll

```
.hero      { height: 300vh; view-timeline: --hero block; }  
.hero-pin  { position: sticky; top: 0; height: 100vh; }
```

The outer `.hero` is 300vh tall — it gives you three viewports' worth of scroll room. The inner `.hero-pin` is 100vh tall (one viewport) and `position: sticky; top: 0`. The result: as you scroll the 300vh runway, the inner pin stays glued to the top of the viewport. Visually nothing moves, but you're advancing through scroll progress. The scroll-driven SVG animation uses that progress to draw the name (see §4–5).

2. The projects TV — pinning a side panel while the timeline scrolls past

```
.projects-grid { display: grid; grid-template-columns: 1fr 1fr; align-items: start; }  
.frame        { position: sticky; top: var(--head-top); }
```

A two-column grid: timeline on the left, TV on the right. The TV's container (`.frame`) sticks to `top: var(--head-top)` (5rem, leaving room for the fixed nav). As long as the parent `.projects-grid` is on screen, the TV stays parked at that offset while the timeline column scrolls past it. Hovering a cassette in the timeline triggers a `:has()` rule that swaps the visible `.tv-pic` inside the still-sticky TV.

Why `align-items: start`? The default grid alignment is `stretch`, which would make the `.frame` grow to the timeline's height. A stretched sticky has no scroll headroom (it fills its row exactly) and can't pin. `align-items: start` lets each column be its natural height.

3. The about portrait — pinning behind scrolling text and layered images

```
.about-stage    { min-height: 220vh; }  
.about-portrait { position: sticky; top: 25%; }
```

The portrait sits in the same grid cell as the white veil, the camera and the text. It's `position: sticky; top: 25%` — pinning a quarter of the viewport down from the top. The runway is 220vh tall, so as you scroll:

1. For the first stretch, the portrait flows with the runway alongside the camera and veil.
2. Once the portrait's natural position reaches 25% from the top, it pins.
3. The veil and camera (not sticky) keep scrolling up and disappear off the top of the viewport.
4. The text (not sticky) scrolls up across the now-uncovered, still-pinned portrait — reading like captions over the trees.
5. Eventually the bottom of `.about-stage` hits 25% from the top, the portrait un-pins, and the whole section scrolls away.

Sticky uses in this project

1. The hero pin (`.hero-pin`)

`position: sticky; top: 0; height: 100vh` inside a `.hero { height: 300vh }` runway. As you scroll the 300vh section, the inner 100vh content "sticks" to the top of the viewport for the duration. This is what gives the scroll-driven SVG animation room to play: you can scroll 300vh while looking at the same fixed-feeling hero, and the scroll progress drives the path reveals.

2. The TV in the projects grid (`.frame`)

`position: sticky; top: var(--head-top)` — the TV stays parked under the nav while the timeline of cassette items scrolls past it. On hover of any timeline item, the corresponding `.tv-pic` swaps in via opacity transition.

3. The about portrait (`.about-portrait`)

`position: sticky; top: 25%` inside a 220vh-tall `.about-stage`. The portrait pins a quarter of the way down the viewport as the white veil/camera/text layers scroll over it.

`position: fixed`

Only one element on the site uses it: `.site-header ( left: 0; right: 0` so it spans exactly the viewport regardless of scroll). Unlike sticky, fixed ignores its containing block and pins to the viewport itself.

Common sticky gotchas

- **An ancestor with `overflow: hidden`** kills sticky in browsers that treat it as a new scroll container. `overflow: clip` doesn't — that's why we use `clip` on `<html>` and `<body>`: it suppresses horizontal scroll without breaking sticky descendants.
- **A flex/grid ancestor with `align-items: stretch`** can make the sticky's container shrink to fit the sticky, so it never has room to "stick." Use `align-items: start` on the parent grid (which is what `.about-stage` does for the portrait/camera/veil layers).
- **Sticky doesn't work without a real `top` (or `bottom` / `left` / `right`)** — without one, it falls back to `position: relative`.

7. At-rules

@font-face

```
@font-face {
  font-family: "BBB Karrik";
  src: url("../fonts/BBB_Karrik.woff2") format("woff2"),
       url("../fonts/BBB_Karrik.woff") format("woff");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}
```

Registers a custom font. `font-display: swap` tells the browser "show fallback text immediately, swap the real font in once it loads." Combined with `<link rel="preload" as="font" crossorigin>` in the document head, the font is requested at the same time as the HTML and arrives before first paint.

@keyframes

```
@keyframes drift {
  0%, 100% { translate: 26vw 0vh; }
  25%      { translate: 0      18vh; }
  50%      { translate: -26vw 0vh; }
  75%      { translate: 0      -18vh; }
}
```

Defines named animation timelines. Then any element binds with `animation: drift 36s linear infinite`. We use keyframes for the hero drift, the vinyl spin, the SVG draw-stroke and a few others.

@media

Conditional rule blocks. Three breakpoints in this site:

- `@media (max-width: 40em)` — phones ($\leq 640\text{px}$). Single-column projects grid, mobile-only TV veil, condensed timeline, smaller about composition.
- `@media (min-width: 40.0625em) and (max-width: 64em)` — small tablets ($\sim 641\text{--}1024\text{px}$). In-between about sizing, narrower hero images grid.

- `@media (prefers-reduced-motion: reduce)` — disables the hero drift, the SVG draw-stroke (just shows the final state), the vinyl spin, etc. Accessibility default.

`@supports`

```
@supports (animation-timeline: view()) {
  /* modern, scroll-driven path */
}
```

Feature query. Only applies the enclosed rules if the browser actually understands the property/value combination. Lets us layer the scroll-driven version on top of a time-based fallback without a JavaScript detect.

8. Recurring techniques

Custom properties as a parameter API

Inline `style="--b: 16px; --t: 48%; --e: 68%"` on each blur span lets one CSS rule render six layered backdrop-filters with different radii and fade points, without any JS. Same idea on the SVG paths (`--i: 8` indexes a stroke into the animation sequence) and the about token (`--about-w` drives camera width, portrait width, veil height and text padding all at once).

Full-bleed inside a constrained `<main>`

```
.about {
  width: 100vw;
  margin-left: calc(50% - 50vw);
  margin-right: calc(50% - 50vw);
}
```

Breaks an element out of `main { max-width: 45rem }` to the full viewport width, recentered. Combined with `html, body { overflow-x: clip }` to prevent the resulting overflow from triggering horizontal scroll.

Layered grid cell + z-index

The about composition stacks portrait, white veil, camera and text in the same grid cell via `grid-area: stage` on all four, then uses `z-index` to control paint order. No absolute positioning needed.

Progressive backdrop-blur

Six `<span>` layers each with `backdrop-filter: blur(var(--b))` at radii `2, 4, 8, 16, 28, 44 px`, each masked by its own linear-gradient. The sum is a smooth blur ramp rather than a single hard-edged blur boundary. Used in the navbar, hero edge fade, and about veil.

`:has()` for parent-aware hover

`.k7-link:has(.player-front:hover) .k7-axis { animation-play-state: running }` — when the cover is hovered, spin the cassette axes; when only the cassette body is hovered, don't. The `:has()` selector lets a parent rule react to a descendant's state without JavaScript.

Mobile-only DOM repurposing via `order`

Same DOM order across all devices. On mobile, `.frame { order: -1 }` pulls the TV above the timeline; combined with `display: grid; grid-template-columns: 1fr` the layout reads top-to-bottom in a sensible way without restructuring the markup.